
Path-Space Mirror Descent for On-Policy Reinforcement Learning under the Generalized Schrödinger Bridge

Yuehu Gong¹ Zeyuan Wang² Yulin Chen² Shutong Ding³ Qingyuan Zhou⁴ Yanwei Fu^{1,5}

¹School of Data Science, Fudan University

²Laboratory for Big Data and Decision, National University of Defense Technology

³ShanghaiTech University

⁴College of Computer Science and Artificial Intelligence, Fudan University

⁵Shanghai Innovation Institute

Abstract

Classical on-policy algorithms such as PPO and mirror descent policy optimization provide stable proximal policy updates through tractable action likelihoods, but are typically instantiated with simple Gaussian policies whose expressiveness can be limited in complex continuous-control tasks. Generative policies based on diffusion and flow models provide more expressive action distributions, but they naturally define distributions over multi-step denoising paths whose terminal action density is often intractable, creating a mismatch with likelihood-based on-policy proximal updates. To address this mismatch, we introduce **GSB-MDPO** (*Generalized Schrödinger Bridge Mirror Descent Policy Optimization*), which formulates on-policy generative policy optimization as a Generalized Schrödinger Bridge problem over state-conditioned generation paths and instantiates the resulting path-measure update through mirror descent policy optimization. The key insight is that the GSB path-space KL plays the role of the proximal term in MDPO while upper-bounding the terminal action KL, enabling direct control of the executed action distribution without explicit terminal action likelihood evaluation. Experiments on 14 continuous-control tasks across Playground and Gym-MuJoCo demonstrate the empirical effectiveness of GSB-MDPO and support path-space regularization as a principled proximal update for multi-step generative policies.

1 Introduction

On-policy reinforcement learning is widely used in continuous control due to its highly parallelizable rollout collection [29, 31]. Methods such as TRPO and PPO stabilize these updates by constraining the policy update through trust regions, likelihood ratios, or KL regularization [39, 41, 44]. Most implementations, however, rely on computationally efficient but restrictive policy classes such as diagonal Gaussians, which can limit performance when effective control requires multimodal action distributions. Recently, generative policies based on diffusion [18, 42] or flow models [15, 23] provide a more expressive alternative. Generative policies have shown promise in planning [19], imitation learning [6], offline RL [34, 49, 50], and online control [17, 20].

Unlike Gaussian policies with tractable action densities, generative policies generate actions as the endpoints of multi-step denoising or transport processes. The resulting terminal action likelihood is often intractable or costly to evaluate [5, 16, 52, 54], making standard on-policy updates based on likelihood ratios or action-space KL regularization difficult to apply. Recent work has sought to make on-policy optimization compatible with generative policies by restoring or approximating the likelihood-ratio terms used in standard proximal updates. DPPO adapts policy-gradient updates to diffusion policies [24, 38], GenPO introduces invertible diffusion policies to recover exact action

likelihoods [9], and FPO constructs a surrogate policy ratio from a conditional flow-matching objective [30].

These methods make on-policy generative RL increasingly practical, but leave open a more fundamental question: *when terminal action likelihoods are intractable, where should proximal regularization be imposed for generative policies?*

A suitable regularizer should be tractable while still controlling the distribution of executed actions. The terminal action density often fails this requirement, whereas the full generation path provides a tractable alternative. Specifically, a generative policy induces a *path measure* over its denoising trajectory, and the KL between path measures upper-bounds the KL between their terminal action marginals. Thus, regularizing the current path measure toward the old one controls the executed policy without explicitly evaluating terminal action likelihoods. This path-space perspective is aligned with generalized Schrödinger bridge formulations, which optimize stochastic path measures relative to reference processes [22, 25].

This leads to *GSB-MDPO*, a path-space mirror descent framework for on-policy generative reinforcement learning. At each iteration, GSB-MDPO improves the old-policy advantage of the terminal action while regularizing the current generation path measure toward the old one through a path-space KL. Both the advantage term and the forward path-KL term can be estimated from old-policy rollout paths via importance sampling, so policy updates do not require additional sampling from the current policy. For stochastic generators with shared diffusion coefficients, the same path-KL further reduces to a drift-MSE regularizer through the standard SDE path-KL identity.

Overall, our contributions are summarized as follows:

- **GSB-MDPO: path-space mirror descent for generative policies.** We introduce *GSB-MDPO*, a mirror-descent policy optimization framework that lifts the proximal update from terminal action distributions to conditional generation path measures under the generalized Schrödinger bridge perspective.
- **Terminal-action control via a path-KL upper bound.** We show that the KL between full generative path measures upper-bounds the KL between terminal action distributions. This allows GSB-MDPO to constrain the executed action distribution through a tractable path-space regularizer, without explicitly evaluating the terminal action likelihood.
- **Empirical validation on continuous-control benchmarks.** We evaluate GSB-MDPO on Playground and Gym-MuJoCo against representative Gaussian and generative on-policy baselines, including PPO, DPPO, FPO, and GenPO. Ablations on path regularization, reference mixing, ratio stabilization, and deterministic evaluation further support the role of path-space regularization.

2 Related Work

2.1 Generative Policies for On-Policy Reinforcement Learning

Recent work has begun to integrate expressive diffusion and flow policies into explicitly on-policy reinforcement learning. DPPO applies policy-gradient fine-tuning to diffusion policies and treats the denoising process as part of the policy optimization procedure [38]. GenPO uses dummy actions to make diffusion policies invertible and likelihood-tractable for PPO-style updates [9–11, 21]. CPPO applies proximal updates to conditional Gaussian denoising transitions, avoiding full denoising likelihoods but benefiting less in unimodal settings [24]. FPO instead avoids exact likelihood computation by using a conditional flow-matching loss to construct a surrogate advantage-weighted ratio for flow policies [30]. ReinFlow injects learnable noise into flow matching policies so that the flow becomes a discrete-time Markov process with tractable likelihoods for online RL fine-tuning [52], while Flow-GRPO studies online policy-gradient training for flow matching models in the GRPO setting [26]. FlowRL further systematizes this area through a modular JAX framework and taxonomy for diffusion/flow-policy RL across Gym, DMC, and IsaacLab benchmarks [13].

GSB-MDPO differs from these approaches in both objective and implementation. Rather than modifying the generative process to recover exact terminal action likelihoods, as in GenPO and ReinFlow, or replacing the likelihood ratio with a flow-matching surrogate, as in FPO, we formulate the update directly as mirror descent over path measures. Both the advantage term and the forward path-KL term are evaluated on old-policy rollout paths through importance sampling. This yields an

on-policy path-space update motivated by the generalized Schrödinger bridge view, while preserving the data-reuse structure of PPO-style training.

2.2 Other Generative Reinforcement Learning Paradigms

A large body of work studies generative policies outside the on-policy setting. In imitation learning and planning, diffusion models have been used to generate trajectories or action sequences from demonstrations and offline data [6, 19]. In offline RL, Diffusion-QL introduced diffusion policies as expressive policy classes regularized by Q-learning objectives [50]. Subsequent work improved the efficiency or compatibility of diffusion policies with offline RL objectives, including IDQL[17], EDP[20], and flow-based Q-learning [12, 32, 34]. These methods highlight the representational benefits of generative policies, but they learn from fixed datasets rather than fresh on-policy rollouts.

Generative policies have also been explored in online and off-policy actor-critic settings. QSM uses Q-score matching to guide diffusion policies with value gradients [36], while QVPO derives a Q-weighted variational policy optimization objective for diffusion-based online RL [8]. DACER extends diffusion policies to online actor-critic learning with entropy regularization [47], and DACERv2 improves diffusion efficiency by adding Q-gradient field supervision and temporal weighting to achieve strong performance with fewer diffusion steps [48]. DIME derives a maximum-entropy objective for diffusion policies via marginal-entropy lower bounds [4]. Related flow-based approaches such as SAC Flow [53] make entropy maximization tractable through velocity reparameterization. FLAC further connects maximum-entropy generative RL with generalized Schrödinger bridges and kinetic-energy regularization [27]. These works are complementary to ours: they mainly study offline, off-policy, or maximum-entropy actor-critic paradigms, whereas GSB-MDPO focuses on on-policy mirror descent over old-policy generative paths.

3 Preliminaries

3.1 On-Policy Mirror Descent Policy Optimization

We consider a discounted Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \rho_0, \gamma)$. A stochastic policy $\pi_\theta(a | s)$ is optimized to maximize

$$J(\pi_\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{h=0}^{\infty} \gamma^h r(s_h, a_h) \right], \quad (1)$$

where h indexes environment interaction time. For policy π , let

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (2)$$

be the advantage function. At iteration k , on-policy methods collect trajectories from the old policy π_k and estimate $A_k(s, a) \approx A^{\pi_k}(s, a)$, often using generalized advantage estimation [40]. Let d_k denote the state visitation distribution induced by π_k .

Mirror descent policy optimization formulates policy improvement as repeatedly solving a regularized local update around the old policy [14, 44]. In its ideal action-space form, the update is

$$\pi_{k+1} = \arg \max_{\pi} \mathbb{E}_{s \sim d_k} \left[\mathbb{E}_{a \sim \pi(\cdot | s)} [A_k(s, a)] - \alpha D_{\text{KL}}(\pi(\cdot | s) \| \pi_k(\cdot | s)) \right], \quad (3)$$

where $\alpha > 0$ controls the proximity to the old policy. For fixed s , the exact nonparametric optimizer is the exponential tilt

$$\pi^*(a | s) = \frac{1}{Z(s)} \pi_k(a | s) \exp \left(\frac{A_k(s, a)}{\alpha} \right), \quad (4)$$

with $Z(s)$ the normalizing constant. In parametric policy classes, MDPO approximately optimizes this KL-regularized objective by gradient updates, retaining the mirror-descent principle while avoiding an exact trust-region solve [44]. GSB-MDPO lifts this action-space update from the terminal action distribution to the full conditional path measure induced by a multi-step generative policy.

3.2 Generalized Schrödinger Bridge for Path-Space Policies

Iterative generative policies define action distributions through stochastic transport processes rather than one-step action densities [18, 23, 42]. Let $t \in [0, 1]$ denote generation time. We model action generation by the SDE

$$da_t = f_\theta(a_t, t, s) dt + \sigma_t dW_t, \quad a_0 \sim \mu_{\text{init}}, \quad (5)$$

where f_θ is a learned drift or velocity field, σ_t is a prescribed noise scale, and W_t is a standard Wiener process. The initial variable a_0 is sampled from a simple prior μ_{init} , and the terminal variable a_1 is the action executed in the environment. Thus, a generative policy induces a path measure over trajectories from the prior endpoint to the action endpoint.

The Generalized Schrödinger Bridge (GSB) formulates stochastic generation as utility-regularized path-measure optimization relative to a reference process [3, 22, 25, 35]. For path-space policy optimization, the GSB objective can be written as

$$P^* = \arg \max_{P: P_0 = \mu_{\text{init}}} \{ \mathbb{E}_{\tau \sim P} [U(a_1)] - \alpha D_{\text{KL}}(P \| P_{\text{ref}}) \}, \quad (6)$$

where P_{ref} is a reference path measure, $P_0 = \mu_{\text{init}}$ fixes the simple prior endpoint, and $U(a_1)$ softly shapes the terminal action distribution instead of prescribing it as a hard marginal constraint. For on-policy generative RL, the reference process is the old path policy and U is the old-policy advantage of the executed action.

In implementation, the continuous process is discretized on a grid

$$0 = t_0 < t_1 < \dots < t_N = 1, \quad (7)$$

with $a^{(n)} \approx a_{t_n}$. A simple Euler transition is

$$a^{(n+1)} = a^{(n)} + (t_{n+1} - t_n) f_\theta(a^{(n)}, t_n, s) + \sigma_{t_n} \sqrt{t_{n+1} - t_n} \epsilon_n, \quad \epsilon_n \sim \mathcal{N}(0, I). \quad (8)$$

For the discrete path

$$\tau := a^{(0:N)} = (a^{(0)}, a^{(1)}, \dots, a^{(N)}), \quad (9)$$

the policy induces the path density

$$P_\theta(a^{(0:N)} | s) = \mu_{\text{init}}(a^{(0)}) \prod_{n=0}^{N-1} p_\theta(a^{(n+1)} | a^{(n)}, s). \quad (10)$$

This GSB view motivates GSB-MDPO in Section 4: mirror descent is lifted from terminal action distributions to generation path measures.

4 Method

The central idea of *GSB-MDPO* is to regard a generative policy not merely as a terminal action distribution, but as a conditional path measure over the full stochastic generation process. This path-space view allows us to define proximal policy improvement directly over generation trajectories while reusing old-policy rollout paths through importance sampling. Our formulation builds on mirror-descent policy optimization [14, 44] and the generalized Schrödinger bridge view of stochastic generation as path-space optimization [22, 25].

4.1 GSB-MDPO: Mirror Descent in Path Space

Let $P_\theta(\tau | s)$ denote the path measure induced by a generative policy, where $\tau = a^{(0:N)}$, $a^{(0)} \sim \mu_{\text{init}}$ is the prior sample, and $a^{(N)}$ is the terminal action executed in the environment. As introduced in Section 3.2, the terminal action distribution is the marginal

$$\pi_\theta(a^{(N)} | s) = \int P_\theta(a^{(0:N)} | s) da^{(0:N-1)}. \quad (11)$$

Thus, for any advantage depending only on the executed action,

$$\mathbb{E}_{a^{(N)} \sim \pi_\theta(\cdot | s)} [A(s, a^{(N)})] = \mathbb{E}_{\tau \sim P_\theta(\cdot | s)} [A(s, a^{(N)})]. \quad (12)$$

Let $P_k := P_{\theta_k}$ be the old path policy used to collect rollouts and let $A_k(s, a^{(N)})$ be the old-policy advantage estimate. GSB-MDPO performs mirror descent over path measures:

$$P_{k+1} = \arg \max_P \mathbb{E}_{s \sim d_k} \left[\mathbb{E}_{\tau \sim P(\cdot | s)} [A_k(s, a^{(N)})] - \alpha D_{\text{KL}}(P(\cdot | s) \| P_k(\cdot | s)) \right], \quad (13)$$

where d_k is the old state distribution and $\alpha > 0$ controls the mirror-descent step size. The reward-improvement term acts on the terminal action, while the proximal regularizer acts on the full generation path.

Proposition 1 (Path KL controls terminal KL). *Let P and Q be path measures over $\tau = a^{(0:N)}$, with terminal marginals π_P and π_Q over $a^{(N)}$. Then*

$$D_{\text{KL}}(P \| Q) \geq D_{\text{KL}}(\pi_P \| \pi_Q) \quad (14)$$

The proof is given in Appendix A.1. This is the KL chain rule, equivalently an instance of the data processing inequality [7]. Related auxiliary-variable bounds appear in variational inference and generative modeling [1, 2, 28, 37, 46], and have recently been used for diffusion-policy entropy bounds [4]. Here, the same structure shows that path-space regularization controls both the terminal marginal and the conditional generation dynamics.

Proposition 2 (Expected old-advantage improvement). *Let $P_{k+1}(\cdot | s)$ solve Eq. (13) for each state s . Then*

$$\mathbb{E}_{s \sim d_k, \tau \sim P_{k+1}(\cdot | s)} [A_k(s, a^{(N)})] \geq \mathbb{E}_{s \sim d_k, \tau \sim P_k(\cdot | s)} [A_k(s, a^{(N)})]. \quad (15)$$

The proof is given in Appendix A.2. This result states that the exact path-space mirror-descent update improves the expected old-policy advantage relative to the old path policy under the fixed old-state distribution.

Proposition 3 (Optimal GSB-MDPO path measure). *For fixed s , the solution of Eq. (13) satisfies*

$$P^*(\tau | s) \propto P_k(\tau | s) \exp\left(\frac{A_k(s, a^{(N)})}{\alpha}\right). \quad (16)$$

Since A_k depends only on the terminal action, this implies

$$\pi^*(a^{(N)} | s) \propto \pi_k(a^{(N)} | s) \exp\left(\frac{A_k(s, a^{(N)})}{\alpha}\right), \quad (17)$$

$$P^*(a^{(0:N-1)} | a^{(N)}, s) = P_k(a^{(0:N-1)} | a^{(N)}, s). \quad (18)$$

A toy visualization of this exponential-tilting solution is provided in Appendix C.1.

The proof is given in Appendix A.3. Proposition 3 shows that the exact update increases the probability of high-advantage terminal actions while preserving the old conditional path structure given the terminal action. This mirrors action-space KL-regularized policy improvement [14, 44], but applies it to path measures rather than one-step action densities.

4.2 Importance-Sampled Path-KL Objective

The exact objective in Eq. (13) involves expectations under P_θ , while on-policy rollouts are collected from P_k . For a stored path τ , define the exact path ratio

$$r_\theta(s, \tau) = \frac{P_\theta(\tau | s)}{P_k(\tau | s)} = \prod_{n=0}^{N-1} \frac{p_\theta(a^{(n+1)} | a^{(n)}, s)}{p_k(a^{(n+1)} | a^{(n)}, s)}. \quad (19)$$

For any path-level function F ,

$$\mathbb{E}_{\tau \sim P_\theta(\cdot | s)} [F(s, \tau)] = \mathbb{E}_{\tau \sim P_k(\cdot | s)} [r_\theta(s, \tau) F(s, \tau)]. \quad (20)$$

Taking $F(s, \tau) = A_k(s, a^{(N)})$ gives the importance-sampled advantage term.

The path KL also admits a drift-MSE form under the SDE view in Section 3.2. For two policies sharing the same initial prior μ_{init} and diffusion coefficient σ_t , with drifts f_θ and f_k , Girsanov’s theorem gives [33]

$$D_{\text{KL}}(P_\theta \| P_k) = \mathbb{E}_{\tau \sim P_\theta} \left[\int_0^1 \frac{\|f_\theta(a_t, t, s) - f_k(a_t, t, s)\|_2^2}{2\sigma_t^2} dt \right], \quad (21)$$

where conditioning on s is omitted when clear. After discretization, the path-KL drift cost is

$$C_{\text{path}}(\theta, \theta_k; s, \tau) = \sum_{n=0}^{N-1} \frac{\Delta t_n}{2\sigma(t_n)^2} \|f_\theta(a^{(n)}, t_n, s) - f_k(a^{(n)}, t_n, s)\|_2^2, \quad \Delta t_n = t_{n+1} - t_n. \quad (22)$$

Thus the MSE-style drift discrepancy is not an ad-hoc penalty; it is the discretized path-KL form for stochastic generators with shared diffusion coefficient. Using Eq. (20), the practical objective folds this cost into an effective advantage:

$$\mathcal{L}_{\text{MDPO}}(\theta) = -\mathbb{E}_{s \sim d_k, \tau \sim P_k} \left[r_\theta(s, \tau) (A_k(s, a^{(N)}) - \lambda C_{\text{path}}(\theta, \theta_k; s, \tau)) \right]. \quad (23)$$

This matches the on-policy mirror-descent objective because both the advantage and path-KL cost are evaluated under the current path measure and then rewritten on old-policy paths by importance sampling. However, the exact path ratio is a product of stepwise likelihood ratios and can have high variance when the current generator moves far from the old one. For this reason, our implementation uses the stabilized path-ratio estimator described in Section 4.4.

4.3 Entropy-Preserving Reference Regularization

In some environments, it is useful to regularize the path policy not only toward the old policy but also toward a simple reference process. Let $P_{\text{ref}}(\cdot | s)$ denote a reference path measure, such as a zero-drift noising process. We consider

$$\max_P \mathbb{E}_{s \sim d_k} \left[\mathbb{E}_{\tau \sim P(\cdot | s)} [A_k(s, a^{(N)})] - \alpha D_{\text{KL}}(P(\cdot | s) \| P_k(\cdot | s)) - \beta D_{\text{KL}}(P(\cdot | s) \| P_{\text{ref}}(\cdot | s)) \right]. \quad (24)$$

The two KL terms combine into a single KL toward

$$P_\eta(\tau | s) = \frac{1}{Z_\eta(s)} P_k(\tau | s)^{1-\eta} P_{\text{ref}}(\tau | s)^\eta, \quad \eta = \frac{\beta}{\alpha + \beta}, \quad (25)$$

since

$$\alpha D_{\text{KL}}(P \| P_k) + \beta D_{\text{KL}}(P \| P_{\text{ref}}) = (\alpha + \beta) D_{\text{KL}}(P \| P_\eta) - (\alpha + \beta) \log Z_\eta(s). \quad (26)$$

Reference-process regularization is common in Schrödinger bridge formulations [22, 25] and has recently appeared in maximum-entropy generative RL [4, 27].

In drift space, completing the square yields the mixed anchor

$$f_\eta = (1 - \eta)f_k + \eta f_{\text{ref}}. \quad (27)$$

In our implementation, f_{ref} is the zero drift, so the per-path cost becomes

$$C_\eta(\theta, \theta_k; s, \tau) = \sum_{n=0}^{N-1} \frac{\Delta t_n}{2\sigma(t_n)^2} \|f_\theta(a^{(n)}, t_n, s) - f_\eta(a^{(n)}, t_n, s)\|_2^2, \quad \Delta t_n = t_{n+1} - t_n. \quad (28)$$

When reference regularization is disabled, $\eta = 0$ and $C_\eta = C_{\text{path}}$.

4.4 Step- and Path-Level Ratio Clipping

The theoretical objective uses the exact path ratio $r_\theta(s, \tau)$. In practice, the path ratio is a product of generation-step likelihood ratios, so numerical errors can accumulate across the generation path. For each generation step, let

$$\Delta \ell_n = \log p_\theta(a^{(n+1)} | a^{(n)}, s) - \log p_k(a^{(n+1)} | a^{(n)}, s), \quad n = 0, \dots, N-1.$$

We use the clipped path ratio

$$\tilde{r}_\theta(s, \tau) = \exp \left[\text{clip} \left(\sum_{n=0}^{N-1} \text{clip}(\Delta \ell_n, -c_{\text{step}}, c_{\text{step}}), -c_{\text{path}}, c_{\text{path}} \right) \right]. \quad (29)$$

The implemented policy loss is

$$\hat{\mathcal{L}}_{\text{MDPO}}(\theta) = -\mathbb{E}_{s \sim d_k, \tau \sim P_k} \left[\tilde{r}_\theta(s, \tau) (A_k(s, a^{(N)}) - \lambda C_\eta(\theta, \theta_k; s, \tau)) \right]. \quad (30)$$

Algorithm 1 GSB-MDPO

Require: Actor θ , critic ϕ , coefficients λ, η , clipping thresholds $c_{\text{step}}, c_{\text{path}}$

```
1: for each on-policy iteration  $k = 0, 1, 2, \dots$  do
2:   Set old actor  $\theta_k \leftarrow \theta$ 
3:   Collect rollouts with  $P_k = P_{\theta_k}$  and store paths, old log-likelihoods, and old drifts
4:   Compute advantages and value targets
5:   for each minibatch do
6:     Recompute current log-likelihoods and drifts on stored paths
7:     Compute path cost  $C_\eta$  using Eq. (28)
8:     Compute clipped path ratio  $\tilde{r}_\theta$  using Eq. (29)
9:     Update actor with Eq. (30)
10:    Update critic  $\phi$  by minimizing  $\mathcal{L}_V(\phi) = \mathbb{E}_{s \sim \mathcal{D}_k} \left[ \left( V_\phi(s) - \hat{V}_k(s) \right)^2 \right]$ 
11:  end for
12: end for
```

The step-level and path-level clipping operations are applied to the accumulated path log-ratio before forming the ratio used in the policy objective. This is a numerical stabilization mechanism for the accumulated path likelihood ratio, not a PPO-style clipped surrogate: PPO clips a terminal action ratio, whereas GSB-MDPO clips stepwise generation log-ratios and their accumulated path log-ratio to control path-ratio variance during minibatch optimization. When the likelihood ratio is clipped, gradients through the ratio are suppressed, but the drift cost $C_\eta(\theta, \theta_k; s, \tau)$ still contributes gradients through the effective advantage term in Eq. (30). Thus, clipping stabilizes the importance weight without removing the path-KL drift regularization signal.

Algorithm 1 summarizes the implementation. All policy updates reuse paths sampled from the old policy; during optimization, we only recompute current path likelihoods and drift values on the stored rollout paths.

5 Experiments

We evaluate GSB-MDPO on two continuous-control benchmark suites: (i) eight Playground tasks from the FPO benchmark [30], built on MuJoCo Playground [51]; and (ii) six Gym-MuJoCo tasks implemented in our on-policy FlowRL-based pipeline [13], using the Gymnasium interface and the MuJoCo physics engine [43, 45]. We compare five methods: Gaussian PPO [41], DPPO [38], FPO [30], GenPO [9], and GSB-MDPO. Within each benchmark, each method uses one fixed hyperparameter configuration across all tasks, without task-specific tuning, except for a memory-related GenPO batch-size adjustment on Humanoid-v5 described in Appendix B. For Gym-MuJoCo, all methods are trained for 40M environment interactions on the same six tasks. Main benchmark curves report the mean over five seeds, with shaded regions showing one standard deviation. Ablation curves report the mean over two seeds, with shaded regions showing the min–max envelope. Implementation details and hyperparameters are provided in Appendix B.

5.1 Main Results on Playground

Figure 1 reports results on the Playground suite. GSB-MDPO obtains competitive or higher returns across the eight tasks, with clear gains on BallInCup, FingerSpin, FingerTurnEasy, and FingerTurnHard. On CheetahRun and the two reaching tasks, multiple methods reach comparable final returns, and GSB-MDPO remains among the leading methods.

These results suggest that the gains are not merely due to using a multi-step generative policy. Compared with DPPO, FPO, and GenPO, GSB-MDPO regularizes the full generation path, which provides a more direct proximal update for on-policy training.

5.2 Main Results on Gym-MuJoCo

Figure 2 reports results on Gym-MuJoCo. GSB-MDPO performs strongly across the six locomotion tasks and is especially effective on higher-dimensional environments such as Ant-v5 and Humanoid-

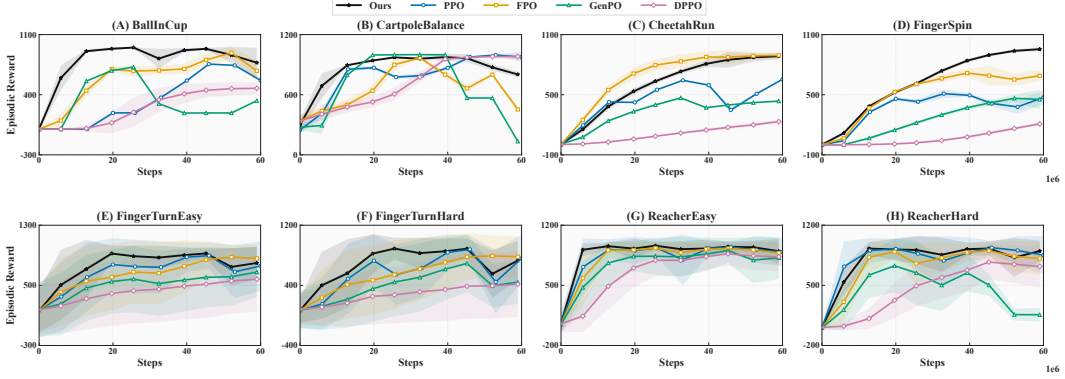


Figure 1: Playground results. Curves show mean $\pm$ one standard deviation over five seeds.

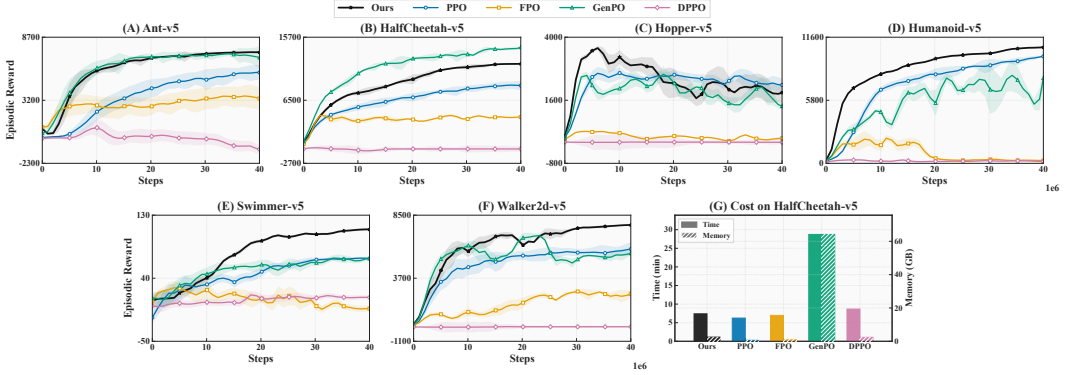


Figure 2: Gym-MuJoCo results. Panels (A)–(F) show learning curves on six tasks, with mean $\pm$ one standard deviation over five seeds. Panel (G) reports representative training time and GPU memory on HalfCheetah-v5.

v5, where both state and action spaces are larger. It also obtains strong final performance on Swimmer-v5 and Walker2d-v5, and remains competitive on Hopper-v5. On HalfCheetah-v5, GenPO achieves the highest return, while GSB-MDPO remains competitive under the same benchmark-level hyperparameter configuration used for all Gym-MuJoCo tasks, reflecting the tradeoff of using one fixed configuration across heterogeneous environments.

Panel (G) shows that GSB-MDPO does not introduce prohibitive training overhead. On HalfCheetah-v5, its training time is close to PPO/FPO/DPPO and substantially lower than GenPO, while its memory usage remains comparable to other path-based generative baselines. Thus, the Gym-MuJoCo results support path-space mirror descent as an effective and practical optimization principle for on-policy generative policies across a separate FlowRL-based training stack.

5.3 Further Analysis

Figure 3 evaluates four components of GSB-MDPO on HalfCheetah-v5. All variants are trained under the same benchmark-level configuration, with only the tested component changed. For the KL and Log-ratio clipping ablation, we compare against the $\eta = 0$ version of GSB-MDPO so that the effect of the path-KL coefficient is isolated from reference mixing.

Path-KL regularization (Fig. 3A). The path-KL coefficient provides the proximal constraint required by the path-space mirror-descent update. Setting this coefficient to zero substantially degrades final performance, indicating that the drift-space penalty is an essential part of the optimization objective rather than an auxiliary stabilizer.

Reference mixing (Fig. 3B). A positive reference-mixing coefficient improves performance over the $\eta = 0$ variant. This indicates that reference mixing acts as an entropy-like regularizer, helping maintain stochasticity in the generative path distribution and preventing premature policy concentration.

Deterministic ODE evaluation (Fig. 3C). This result supports using the deterministic path at evaluation time: the policy is trained with stochastic generation paths, while evaluation removes sampling noise by following the learned deterministic drift.

Log-ratio clipping (Fig. 3D). Step- and path-level log-ratio clipping improves the stability of importance-sampled path-ratio optimization. Without clipping, accumulated generation-step likelihood ratios produce a less reliable update.

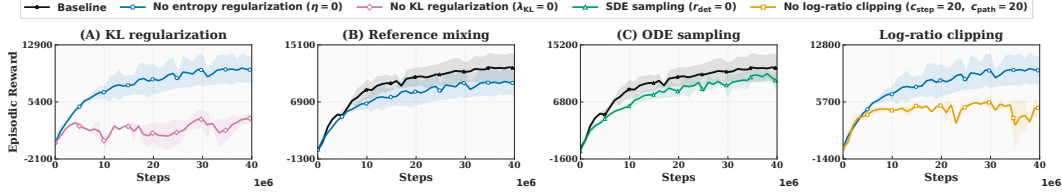


Figure 3: Ablations on HalfCheetah-v5. Curves show the mean and min–max envelope over two seeds.

Sensitivity to Generation Steps. We additionally study the sensitivity of GSB-MDPO to the number of generation steps. Figure 4 reports results on HalfCheetah-v5 with 4, 8, 16, and 32 flow steps. All variants are trained for 40M environment interactions and averaged over two random seeds.

The results show that GSB-MDPO is not highly sensitive to the exact number of generation steps on this task. All four settings reach similar final performance by the end of training. Using more steps can slightly smooth the later-stage improvement, while fewer steps such as 4 or 8 remain competitive and sometimes learn faster early in training. This suggests that the path-space MDPO objective is compatible with both short and moderately long generative paths, and that reducing the number of generation steps may be a practical way to trade off sampling cost and performance.

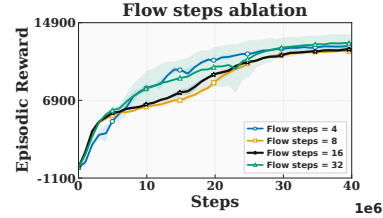


Figure 4: Ablation of the number of generation steps for GSB-MDPO on HalfCheetah-v5. Solid curves show the mean over two seeds, and shaded regions show the min–max envelope across seeds.

6 Conclusion and Limitations

GSB-MDPO connects generalized Schrödinger bridge modeling with mirror descent policy optimization to derive a path-space proximal update for multi-step generative policies. Instead of relying on generally intractable terminal action densities, it regularizes the conditional generation path measure. The resulting path KL controls the terminal action KL, can be estimated from on-policy data, and reduces to a drift-MSE penalty under shared diffusion coefficients. Despite optimizing multi-step generation paths, GSB-MDPO achieves PPO-comparable efficiency by avoiding additional current-policy sampling. Experiments on Playground and Gym-MuJoCo, together with ablations, support the effectiveness of path-space regularization for on-policy generative reinforcement learning.

Despite its strong performance, GSB-MDPO currently uses a fixed scalar noise schedule shared across action dimensions. Although effective on normalized continuous-control benchmarks, this design may be suboptimal for heterogeneous actuators that require state-dependent stochasticity, a limitation also noted in related bridge-based maximum-entropy methods [27]. Future work should explore adaptive noise schedules and fewer-step path-space updates for broader control domains.

References

- [1] Felix V. Agakov and David Barber. An auxiliary variational method. In *International Conference on Neural Information Processing (ICONIP)*, pages 561–566. Springer, 2004.
- [2] Oleg Arenz, Gerhard Neumann, and Mingjun Zhong. Efficient gradient-free variational inference using policy search. In *International Conference on Machine Learning (ICML)*, pages 234–243, 2018.
- [3] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the monge-kantorovich mass transfer problem. *Numerische Mathematik*, 84:375–393, 2000.
- [4] Onur Celik, Zechu Li, Denis Blessing, Ge Li, Daniel Palenicek, Jan Peters, Georgia Chalvatzaki, and Gerhard Neumann. DIME: Diffusion-based maximum entropy reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 6958–6977, 2025.
- [5] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [6] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2025.
- [7] Thomas M. Cover. *Elements of Information Theory*. John Wiley & Sons, 1999.
- [8] Shutong Ding, Ke Hu, Zhenhao Zhang, Kan Ren, Weinan Zhang, Jingyi Yu, Jingya Wang, and Ye Shi. Diffusion-based reinforcement learning via q-weighted variational policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [9] Shutong Ding, Ke Hu, Shan Zhong, Haoyang Luo, Weinan Zhang, Jingya Wang, Jun Wang, and Ye Shi. Genpo: Generative diffusion models meet on-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [10] Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. In *International Conference on Learning Representations Workshop (ICLR Workshop)*, 2015.
- [11] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real NVP. In *International Conference on Learning Representations (ICLR)*, 2017.
- [12] Nicolas Espinosa-Dice, Yiyi Zhang, Yiding Chen, Bradley Guo, Owen Oertell, Gokul Swamy, Kianté Brantley, and Wen Sun. Scaling offline rl via efficient and expressive shortcut models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [13] Chenxiao Gao, Edward Chen, Tianyi Chen, and Bo Dai. FlowRL: A taxonomy and modular framework for reinforcement learning with diffusion policies. *arXiv preprint arXiv:2603.27450*, 2026.
- [14] Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized markov decision processes. In *International Conference on Machine Learning (ICML)*, 2019.
- [15] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. Oral.
- [16] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFLORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations (ICLR)*, 2019.
- [17] Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- [18] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [19] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning (ICML)*, pages 9902–9915, 2022.
- [20] Bingyi Kang, Xiao Ma, Chao Du, Tianyu Pang, and Shuicheng Yan. Efficient diffusion policies for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [22] Christian Léonard. A survey of the schrödinger problem and some of its connections with optimal transport. *Discrete and Continuous Dynamical Systems*, 34(4):1533–1574, 2014.
- [23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [24] Ben Liu, Shunpeng Yang, and Hua Chen. Diffusion policy through conditional proximal policy optimization. *arXiv preprint arXiv:2603.04790*, 2026.
- [25] Guan-Hong Liu, Yaron Lipman, Maximilian Nickel, Brian Karrer, Evangelos A. Theodorou, and Ricky T. Q. Chen. Generalized schrödinger bridge matching. In *International Conference on Learning Representations (ICLR)*, 2024.
- [26] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online rl. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [27] Lei Lv, Yunfei Li, Yu Luo, Fuchun Sun, and Xiao Ma. FLAC: Maximum entropy rl via kinetic energy regularized bridge matching. *arXiv preprint arXiv:2602.12829*, 2026.
- [28] Lars Maaløe, Casper Kaae Sønderby, Søren Kaae Sønderby, and Ole Winther. Auxiliary deep generative models. In *International Conference on Machine Learning (ICML)*, pages 1445–1453, 2016.
- [29] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.
- [30] David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients. In *International Conference on Learning Representations (ICLR)*, 2026.
- [31] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning (ICML)*, 2016.
- [32] Zhancun Mu. Deflow: Decoupling manifold modeling and value maximization for offline policy extraction. *arXiv preprint arXiv:2601.10471*, 2026.
- [33] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer, 6 edition, 2003.
- [34] Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. In *International Conference on Machine Learning (ICML)*, pages 48104–48127, 2025.
- [35] Michele Pavon, Giulio Trigila, and Esteban G. Tabak. The data-driven schrödinger bridge. *Communications on Pure and Applied Mathematics*, 74(7):1545–1573, 2021.
- [36] Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. Learning a diffusion model policy from rewards via q-score matching. In *International Conference on Machine Learning (ICML)*, pages 41163–41182, 2024.

- [37] Rajesh Ranganath, Dustin Tran, and David M. Blei. Hierarchical variational models. In *International Conference on Machine Learning (ICML)*, pages 324–333, 2016.
- [38] Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations (ICLR)*, 2025.
- [39] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [40] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [41] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [42] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- [43] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.
- [44] Manan Tomar, Lior Shani, Yonathan Efroni, and Mohammad Ghavamzadeh. Mirror descent policy optimization. In *International Conference on Learning Representations (ICLR)*, 2022.
- [45] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [46] Dustin Tran, David M. Blei, and Edoardo M. Airolidi. Copula variational inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 3564–3572, 2015.
- [47] Yinuo Wang, Likun Wang, Yuxuan Jiang, Wenjun Zou, Tong Liu, Xujie Song, Wenxuan Wang, Liming Xiao, Jiang Wu, Jingliang Duan, and Shengbo Eben Li. Diffusion actor-critic with entropy regulator. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [48] Yinuo Wang, Likun Wang, Mining Tan, Wenjun Zou, Xujie Song, Wenxuan Wang, Tong Liu, Guojian Zhan, Tianze Zhu, Shiqi Liu, Zeyu He, Feihong Zhang, Jingliang Duan, and Shengbo Eben Li. Enhanced DACER algorithm with high diffusion efficiency. *arXiv preprint arXiv:2505.23426*, 2025.
- [49] Zeyuan Wang, Da Li, Yulin Chen, Ye Shi, Liang Bai, Tianyuan Yu, and Yanwei Fu. One-step generative policies with q-learning: A reformulation of meanflow. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- [50] Zhendong Wang, Jonathan J. Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023.
- [51] Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A Kahrs, et al. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.
- [52] Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

- [53] Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Mo Guang, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. *International Conference on Learning Representations (ICLR)*, 2026.
- [54] Yixian Zhang, Shu'ang Yu, Tonghe Zhang, Guang Mo, Haojia Hui, Kaiwen Long, Yu Wang, Chao Yu, and Wenbo Ding. SAC flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. In *International Conference on Learning Representations (ICLR)*, 2026.

A Proofs

A.1 Proof of the Path-KL Decomposition

Let P and Q be path measures over $\tau = a^{(0:N)}$. Factorizing both measures by the terminal action gives

$$P(a^{(0:N)}) = \pi_P(a^{(N)})P(a^{(0:N-1)} \mid a^{(N)}), \quad (31)$$

$$Q(a^{(0:N)}) = \pi_Q(a^{(N)})Q(a^{(0:N-1)} \mid a^{(N)}). \quad (32)$$

Then

$$D_{\text{KL}}(P \parallel Q) = \mathbb{E}_{\tau \sim P} \left[\log \frac{P(a^{(0:N)})}{Q(a^{(0:N)})} \right] \quad (33)$$

$$= \mathbb{E}_{\tau \sim P} \left[\log \frac{\pi_P(a^{(N)})}{\pi_Q(a^{(N)})} + \log \frac{P(a^{(0:N-1)} \mid a^{(N)})}{Q(a^{(0:N-1)} \mid a^{(N)})} \right] \quad (34)$$

$$= D_{\text{KL}}(\pi_P \parallel \pi_Q) + \mathbb{E}_{a^{(N)} \sim \pi_P} \left[D_{\text{KL}} \left(P(a^{(0:N-1)} \mid a^{(N)}) \parallel Q(a^{(0:N-1)} \mid a^{(N)}) \right) \right] \quad (35)$$

$$\geq D_{\text{KL}}(\pi_P \parallel \pi_Q). \quad (36)$$

A.2 Proof of Expected Old-Advantage Improvement

For fixed s , define

$$\mathcal{J}_s(P) = \mathbb{E}_{\tau \sim P(\cdot \mid s)} [A_k(s, a^{(N)})] - \alpha D_{\text{KL}}(P(\cdot \mid s) \parallel P_k(\cdot \mid s)). \quad (37)$$

Since P_{k+1} maximizes $\mathcal{J}_s$ and P_k is feasible,

$$\mathbb{E}_{\tau \sim P_{k+1}} [A_k(s, a^{(N)})] - \alpha D_{\text{KL}}(P_{k+1} \parallel P_k) = \mathcal{J}_s(P_{k+1}) \quad (38)$$

$$\geq \mathcal{J}_s(P_k) \quad (39)$$

$$= \mathbb{E}_{\tau \sim P_k} [A_k(s, a^{(N)})]. \quad (40)$$

Therefore,

$$\mathbb{E}_{\tau \sim P_{k+1}} [A_k(s, a^{(N)})] \geq \mathbb{E}_{\tau \sim P_k} [A_k(s, a^{(N)})] + \alpha D_{\text{KL}}(P_{k+1} \parallel P_k) \quad (41)$$

$$\geq \mathbb{E}_{\tau \sim P_k} [A_k(s, a^{(N)})]. \quad (42)$$

Taking expectation over $s \sim d_k$ gives the state-averaged result.

A.3 Proof of the Optimal GSB-MDPO Path Measure

Fix s and write $\tau = a^{(0:N)}$. The per-state optimization problem is

$$\max_P \int P(\tau) A_k(s, a^{(N)}) d\tau - \alpha \int P(\tau) \log \frac{P(\tau)}{P_k(\tau)} d\tau, \quad (43)$$

$$\text{s.t.} \quad \int P(\tau) d\tau = 1. \quad (44)$$

Introducing a Lagrange multiplier λ gives

$$\mathcal{L}(P, \lambda) = \int P(\tau) A_k(s, a^{(N)}) d\tau - \alpha \int P(\tau) \log \frac{P(\tau)}{P_k(\tau)} d\tau \quad (45)$$

$$+ \lambda \left(\int P(\tau) d\tau - 1 \right). \quad (46)$$

The first-order condition and its rearranged form are

$$\frac{\delta \mathcal{L}}{\delta P(\tau)} = A_k(s, a^{(N)}) - \alpha \left(\log \frac{P^*(\tau)}{P_k(\tau)} + 1 \right) + \lambda = 0, \quad (47)$$

$$\log \frac{P^*(\tau)}{P_k(\tau)} = \frac{A_k(s, a^{(N)})}{\alpha} + \frac{\lambda}{\alpha} - 1. \quad (48)$$

Absorbing the constant $\exp(\lambda/\alpha - 1)$ into the normalizer yields

$$P^*(\tau | s) = \frac{1}{Z(s)} P_k(\tau | s) \exp \left(\frac{A_k(s, a^{(N)})}{\alpha} \right), \quad (49)$$

$$Z(s) = \int P_k(\tau | s) \exp \left(\frac{A_k(s, a^{(N)})}{\alpha} \right) d\tau. \quad (50)$$

Equivalently,

$$Z(s) = \mathbb{E}_{\tau \sim P_k(\cdot | s)} \left[\exp \left(\frac{A_k(s, a^{(N)})}{\alpha} \right) \right]. \quad (51)$$

Since the tilt depends only on the terminal action $a^{(N)}$, the joint optimizer factorizes as

$$P^*(a^{(0:N)} | s) = \frac{1}{Z(s)} \pi_k(a^{(N)} | s) P_k(a^{(0:N-1)} | a^{(N)}, s) \exp \left(\frac{A_k(s, a^{(N)})}{\alpha} \right), \quad (52)$$

$$\pi^*(a^{(N)} | s) = \int P^*(a^{(0:N)} | s) da^{(0:N-1)} \quad (53)$$

$$= \frac{1}{Z(s)} \pi_k(a^{(N)} | s) \exp \left(\frac{A_k(s, a^{(N)})}{\alpha} \right), \quad (54)$$

$$P^*(a^{(0:N-1)} | a^{(N)}, s) = \frac{P^*(a^{(0:N)} | s)}{\pi^*(a^{(N)} | s)} \quad (55)$$

$$= P_k(a^{(0:N-1)} | a^{(N)}, s). \quad (56)$$

A.4 Importance-Sampling Identities

For any path-level function $F(s, \tau)$,

$$\mathbb{E}_{\tau \sim P_\theta(\cdot | s)} [F(s, \tau)] = \int P_\theta(\tau | s) F(s, \tau) d\tau \quad (57)$$

$$= \int P_k(\tau | s) \frac{P_\theta(\tau | s)}{P_k(\tau | s)} F(s, \tau) d\tau \quad (58)$$

$$= \mathbb{E}_{\tau \sim P_k(\cdot | s)} [r_\theta(s, \tau) F(s, \tau)], \quad (59)$$

which recovers Eq. (20). Taking $F(s, \tau) = A_k(s, a^{(N)})$ gives the importance-sampled advantage term. Similarly,

$$D_{\text{KL}}(P_\theta(\cdot | s) \| P_k(\cdot | s)) = \int P_\theta(\tau | s) \log \frac{P_\theta(\tau | s)}{P_k(\tau | s)} d\tau \quad (60)$$

$$= \int P_k(\tau | s) \frac{P_\theta(\tau | s)}{P_k(\tau | s)} \log \frac{P_\theta(\tau | s)}{P_k(\tau | s)} d\tau \quad (61)$$

$$= \mathbb{E}_{\tau \sim P_k(\cdot | s)} [r_\theta(s, \tau) \log r_\theta(s, \tau)]. \quad (62)$$

A.5 Path KL as Drift MSE

Consider two path measures P_θ and P_k induced by SDEs with the same initial prior μ_{init} and the same diffusion coefficient:

$$da_t = f_\theta(a_t, t, s) dt + \sigma_t dW_t, \quad a_0 \sim \mu_{\text{init}}, \quad (63)$$

$$da_t = f_k(a_t, t, s) dt + \sigma_t dW_t, \quad a_0 \sim \mu_{\text{init}}. \quad (64)$$

Under the standard absolute-continuity assumptions of Girsanov's theorem,

$$D_{\text{KL}}(P_\theta(\cdot | s) \| P_k(\cdot | s)) = \mathbb{E}_{\tau \sim P_\theta(\cdot | s)} \left[\int_0^1 \frac{\|f_\theta(a_t, t, s) - f_k(a_t, t, s)\|_2^2}{2\sigma_t^2} dt \right]. \quad (65)$$

No initial KL term appears because the two path measures share the same initial prior. A first-order discretization over the grid $0 = t_0 < \dots < t_N = 1$ gives

$$\int_0^1 \frac{\|f_\theta(a_t, t, s) - f_k(a_t, t, s)\|_2^2}{2\sigma_t^2} dt \approx \sum_{n=0}^{N-1} \frac{\Delta t_n}{2\sigma(t_n)^2} \|f_\theta(a^{(n)}, t_n, s) - f_k(a^{(n)}, t_n, s)\|_2^2 \quad (66)$$

$$= C_{\text{path}}(\theta, \theta_k; s, \tau), \quad (67)$$

where $\Delta t_n = t_{n+1} - t_n > 0$. This yields the drift-MSE path regularizer in Eq. (22).

A.6 Composite KL and Mixed-Anchor Drift Penalty

For fixed s , write all path measures as conditional measures given s . Let

$$\eta = \frac{\beta}{\alpha + \beta}, \quad (68)$$

$$P_\eta(\tau | s) = \frac{1}{Z_\eta(s)} P_k(\tau | s)^{1-\eta} P_{\text{ref}}(\tau | s)^\eta. \quad (69)$$

Then

$$\log P_\eta(\tau | s) = (1 - \eta) \log P_k(\tau | s) + \eta \log P_{\text{ref}}(\tau | s) - \log Z_\eta(s). \quad (70)$$

Therefore,

$$\alpha D_{\text{KL}}(P(\cdot | s) \| P_k(\cdot | s)) + \beta D_{\text{KL}}(P(\cdot | s) \| P_{\text{ref}}(\cdot | s)) \quad (71)$$

$$= (\alpha + \beta) \mathbb{E}_{\tau \sim P(\cdot | s)} [\log P(\tau | s) - (1 - \eta) \log P_k(\tau | s) - \eta \log P_{\text{ref}}(\tau | s)] \quad (72)$$

$$= (\alpha + \beta) \mathbb{E}_{\tau \sim P(\cdot | s)} \left[\log \frac{P(\tau | s)}{P_\eta(\tau | s)} \right] - (\alpha + \beta) \log Z_\eta(s) \quad (73)$$

$$= (\alpha + \beta) D_{\text{KL}}(P(\cdot | s) \| P_\eta(\cdot | s)) - (\alpha + \beta) \log Z_\eta(s). \quad (74)$$

For fixed s , $Z_\eta(s)$ is independent of the optimized conditional path measure $P(\cdot | s)$. Since the mirror-descent update takes the outer expectation over the fixed old-policy state distribution d_k , the normalizer contributes a policy-independent constant to the fixed- d_k objective.

For fixed path samples and shared diffusion coefficient, the pointwise weighted drift discrepancy satisfies

$$\alpha \|f_\theta - f_k\|_2^2 + \beta \|f_\theta - f_{\text{ref}}\|_2^2 \quad (75)$$

$$= (\alpha + \beta) \left\| f_\theta - \frac{\alpha f_k + \beta f_{\text{ref}}}{\alpha + \beta} \right\|_2^2 + \frac{\alpha \beta}{\alpha + \beta} \|f_k - f_{\text{ref}}\|_2^2. \quad (76)$$

With

$$f_\eta = (1 - \eta) f_k + \eta f_{\text{ref}}, \quad (77)$$

the first term gives the mixed-anchor drift penalty. When the drift loss is evaluated on fixed rollout paths, the residual term in Eq. (76) does not depend on the optimized drift f_θ and can be omitted from the practical surrogate. Discretizing the resulting drift discrepancy over $0 = t_0 < \dots < t_N = 1$ gives Eq. (28).

B Implementation Details

B.1 Hardware Configurations

All experiments were carried out on a server equipped with two AMD EPYC 7763 64-Core processors. Each socket contains 64 physical cores with 2 hardware threads per core, yielding up to 256 logical CPUs. The maximum CPU frequency is approximately 3.53 GHz. The system has a two-node NUMA topology, with CPUs distributed across the two sockets.

For GPU acceleration, we used 8 NVIDIA A800 80GB PCIe GPUs, each with 80 GB of memory. The GPUs were operated with NVIDIA driver version 570.211.01 and CUDA 12.8. Multi-Instance GPU (MIG) was disabled during the experiments. The machine is equipped with 2.0 TiB of system memory and runs Ubuntu 24.04.3 LTS with Linux kernel 6.8.0.

B.2 Codebases and Experimental Frameworks

Our experiments are implemented on top of two public codebases. For the Playground experiments, we use the official FPO repository, whose `playground/` implementation provides a JAX-based training pipeline for MuJoCo Playground and Brax [30]. The FPO codebase already supports PPO, FPO, and DPPO within the same Playground training interface; DPPO is implemented as one mode of the FPO training script. We reproduce these three baselines using the original Playground implementation and hyperparameter structure. We further implement GenPO in the same FPO Playground pipeline by adapting the corresponding GenPO implementation logic from FlowRL. Our GSB-MDPO agent is added to this same pipeline, so all Playground comparisons share the same environment interface, rollout collection procedure, training loop, and evaluation protocol.

For the Gym-MuJoCo experiments, we build on FlowRL, a modular JAX framework for reinforcement learning with diffusion and flow policies [13]. FlowRL provides implementations of PPO, DPPO, FPO, and GenPO, together with standardized infrastructure for diffusion/flow-policy RL. The original FlowRL codebase includes on-policy IsaacLab launch scripts and off-policy Gym-MuJoCo launch scripts. Since our Gym-MuJoCo comparison is on-policy, we implement an additional on-policy Gym-MuJoCo launcher following the FlowRL training interface. We also implement GSB-MDPO in this FlowRL-based Gym-MuJoCo pipeline. Due to interface differences between FPO Playground and FlowRL/Gym-MuJoCo, the Playground and Gym-MuJoCo implementations of GSB-MDPO are maintained separately, but they share the same core algorithmic logic: old-policy path rollout storage, current path likelihood recomputation, importance-sampled MDPO loss, and path-KL drift regularization.

For Gym-MuJoCo, we use the IsaacLab on-policy configurations from FlowRL as the starting point for network architectures and algorithmic hyperparameters, since the Gym-MuJoCo locomotion tasks are structurally similar to the corresponding IsaacLab locomotion tasks. This includes policy and value network architectures, optimizer settings, rollout/update settings, and generative-policy parameters whenever applicable. The main systematic difference is the total number of environment interactions: FlowRL’s IsaacLab setting uses 100M environment steps with massively parallel GPU simulation, whereas our Gym-MuJoCo experiments use 40M steps because Gym-MuJoCo does not support the same scale of GPU-parallel environment interaction.

B.3 Hyperparameters

For each benchmark, we use a single benchmark-level hyperparameter configuration for each method across all tasks, without task-specific tuning. This protocol is intended to make the comparison depend on the algorithmic design rather than per-environment hyperparameter search. In the Gym-MuJoCo benchmark, GSB-MDPO uses a cosine learning-rate schedule for the policy/flow optimizer, decaying from the reported initial learning rate to zero over training.

Playground. Table 1 summarizes the hyperparameters used for the Playground experiments. The PPO, FPO, and DPPO configurations follow the FPO Playground setup [30]; DPPO is implemented through the denoising-MDP mode of the FPO training pipeline. GenPO and GSB-MDPO are implemented in the same Playground training interface for fair comparison.

Gym-MuJoCo. Table 2 summarizes the hyperparameters used for the Gym-MuJoCo experiments. For each method, we use a single hyperparameter configuration across all six Gym-MuJoCo tasks, without task-specific tuning. The Gym-MuJoCo configurations are adapted from the FlowRL-style on-policy IsaacLab configurations, with an additional on-policy Gym-MuJoCo launcher implemented for our experiments.

Table 1: Hyperparameters for Playground experiments. A single configuration is used for all Playground tasks within each method.

Hyperparameter	PPO	DPPO	FPO	GenPO	GSB-MDPO
<i>Benchmark and rollout setup</i>					
Total environment steps	60M	60M	60M	60M	60M
Number of environments	2048	2048	2048	2048	2048
Episode length	1000	1000	1000	1000	1000
Number of evaluations	10	10	10	10	10
Unroll length	30	30	30	30	30
Batch size	1024	1024	1024	1024	1024
Number of minibatches	32	32	32	32	32
Updates per batch	16	16	16	16	16
Discount factor γ	0.995 [†]	0.995 [†]	0.995 [†]	0.995 [†]	0.995 [†]
GAE λ	0.95	0.95	0.95	0.95	0.95
Reward scaling	10.0	10.0	10.0	10.0	10.0
Normalize observations	Yes	Yes	Yes	Yes	Yes
Normalize advantages	Yes	Yes	Yes	Yes	Yes
Optimizer	Adam	Adam	Adam	Adam	Adam
<i>Actor configuration</i>					
Actor network	MLP, 4×32	MLP, 4×32	MLP, 4×32	MLP, 3×64	MLP, 4×32
Actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	1×10^{-3}
Entropy coefficient	0	0	0	0	0
PPO clip ϵ	0.1	0.2	0.05	0.2	–
<i>Critic configuration</i>					
Critic network	MLP, 5×256	MLP, 5×256	MLP, 5×256	MLP, 5×256	MLP, 5×256
Critic learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}	1×10^{-3}
Value loss coefficient	0.25	0.25	0.25	1.0	0.1
<i>Generative-policy configuration</i>					
Generation steps	–	10	10	5	8
Time embedding dimension	–	8	8	32	8
Policy output scale	–	0.25	0.25	–	0.25
Sigma schedule	–	Constant	–	–	Exp.
$\sigma_{\max} / \sigma_{\min}$	–	0.05 / 0.05	–	–	3.0 / 1.0
<i>FPO-specific configuration</i>					
Samples per action	–	–	8	–	–
<i>GenPO-specific configuration</i>					
Flow mix parameter	–	–	–	0.9	–
Compress coefficient	–	–	–	0.01	–
<i>GSB-MDPO-specific configuration</i>					
Step log-ratio clip	–	–	–	–	0.1
Path log-ratio clip	–	–	–	–	0.3
KL penalty coefficient	–	–	–	–	0.8
Reference mix η	–	–	–	–	0.0

[†] Following the FPO Playground setup, $\gamma = 0.995$ is used for most tasks; BallInCup and FingerSpin use $\gamma = 0.95$.

Table 2: Hyperparameters for Gym-MuJoCo experiments. A single configuration is used for all Gym-MuJoCo tasks within each method.

Hyperparameter	PPO	DPPO	FPO	GenPO	GSB-MDPO
<i>Benchmark and rollout setup</i>					
Total environment steps	40M	40M	40M	40M	40M
Evaluation interval	1M	1M	1M	1M	1M
Evaluation episodes	10	10	10	10	10
Number of environments	1024	1024	1024	1024	1024
Rollout length	24	24	24	24	24
Batch size	6144	6144	6144	6144 [‡]	6144
Number of minibatches	4	4	4	4 [‡]	4
Training epochs	4	4	4	4	4
Discount factor γ	0.99	0.99	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95	0.95	0.95
Reward scaling	1.0	1.0	1.0	1.0	1.0
Normalize observations	Yes	Yes	Yes	Yes	Yes
Normalize advantages	Yes	Yes	Yes	Yes	Yes
Gradient clip norm	1.0	1.0	1.0	1.0	1.0
<i>Actor configuration</i>					
Actor network	MLP, 3×256	MLP, 3×256	MLP, 3×256	MLP, 3×256	MLP, 3×256
Actor activation	ELU	SiLU	SiLU	SiLU	SiLU
Actor learning rate	1×10^{-4}	1×10^{-3}	1×10^{-4}	1×10^{-3}	7.5×10^{-4}
PPO clip ϵ	0.2	0.2	0.05	0.2	–
Entropy coefficient	0	–	–	0.0	–
<i>Critic configuration</i>					
Critic network	MLP, 3×256	MLP, 3×256	MLP, 3×256	MLP, 3×256	MLP, 3×256
Critic activation	ELU	ELU	ELU	ELU	ELU
Critic learning rate	1×10^{-3}	1×10^{-3}	1×10^{-3}	1×10^{-3}	1×10^{-3}
<i>Generative-policy configuration</i>					
Generation steps	–	10	10	16	16
Flow / diffusion hidden dims	–	3×256	3×256	3×256	3×256
Time embedding dimension	–	32	16	16	16
Flow / diffusion output scale	–	–	0.25	0.25	0.25
Noise schedule	–	Cosine DDPM	–	–	Linear
Min log-prob std.	–	0.1	–	–	–
$\sigma_{\max} / \sigma_{\min}$	–	–	–	–	3.0 / 0.3
<i>FPO-specific configuration</i>					
Monte Carlo samples	–	–	8	–	–
Additive noise	–	–	0.1	–	–
<i>GenPO-specific configuration</i>					
Flow mix parameter	–	–	–	0.9	–
Compress coefficient	–	–	–	0.01	–
<i>GSB-MDPO-specific configuration</i>					
Step log-ratio clip	–	–	–	–	0.1
Path log-ratio clip	–	–	–	–	0.4
KL penalty coefficient	–	–	–	–	0.08
Reference mix η	–	–	–	–	0.02

[‡] For GenPO on Humanoid-v5 only, we use batch size 4096 and 6 minibatches due to GPU memory constraints. All other GenPO Gym-MuJoCo tasks use the default batch size 6144 and 4 minibatches.

C Additional Experimental Results

C.1 Toy Analysis of the Optimal Path-Space Update

We include a controlled two-dimensional toy example to visualize the closed-form optimizer in Proposition 3. The old policy p_{old} is a four-mode Gaussian mixture over the terminal action $x = (x_1, x_2)$. We define a quadrant-wise preference $A(x)$ and construct the exponential-tilted target

$$p^*(x) \propto p_{\text{old}}(x) \exp\left(\frac{A(x)}{\beta}\right), \quad (78)$$

which is the terminal marginal implied by the optimal path-space mirror-descent update. The preference assigns the largest weight to quadrant QIII and smaller weights to the other quadrants, so the optimal update should increase probability mass in QIII while preserving the local mode structure inherited from the old policy.

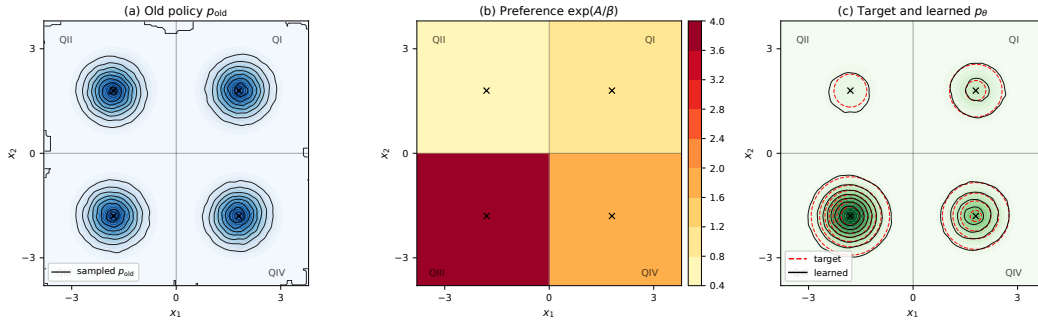


Figure 5: Toy visualization of the optimal GSB-MDPO update. Left: old policy p_{old} with four approximately balanced modes. Middle: quadrant-wise preference weights $\exp(A/\beta)$. Right: target exponential-tilted marginal p^* and the learned distribution p_θ . The learned policy closely matches the theoretical tilted target, increasing mass in the preferred quadrant while preserving the old policy’s mode structure.

Figure 5 confirms the behavior predicted by Proposition 3. The exponential tilting changes the relative mass of the modes according to the terminal preference while leaving the conditional path structure unchanged in the ideal solution. In the learned distribution, the quadrant masses closely match the target distribution, with a quadrant-mass ℓ_1 error of 0.100. This illustrates that GSB-MDPO implements the intended mirror-descent update: it does not simply collapse to the highest-reward mode, but reweights the old generative policy according to the advantage-induced exponential tilt.

C.2 Step Log-Ratio Clipping Analysis

To better understand the role of step-level clipping, we record the fraction of rollout samples whose step log-ratio exceeds the clipping threshold during training. Figure 6 reports this diagnostic on HalfCheetah-v5 for the full GSB-MDPO baseline, averaged over two random seeds.

The clipping fraction rises quickly at the beginning of training and then stabilizes at roughly 0.2 throughout most of optimization. This behavior suggests that step-level clipping is neither a rarely activated corner case nor a mechanism that completely dominates learning. Instead, it acts as a persistent stabilizer for the importance-sampled path-ratio estimator, especially during periods when the current policy moves away from the old one.

The diagnostic also helps clarify the role of clipping in GSB-MDPO. The path-KL drift regularizer is part of the mirror-descent objective itself, whereas step log-ratio clipping is an implementation device for controlling the variance of the accumulated path likelihood ratio. The moderate and stable clipping frequency observed in Figure 6 indicates that this mechanism is active often enough to matter in practice, while still leaving most stepwise likelihood ratios within the unclipped regime.

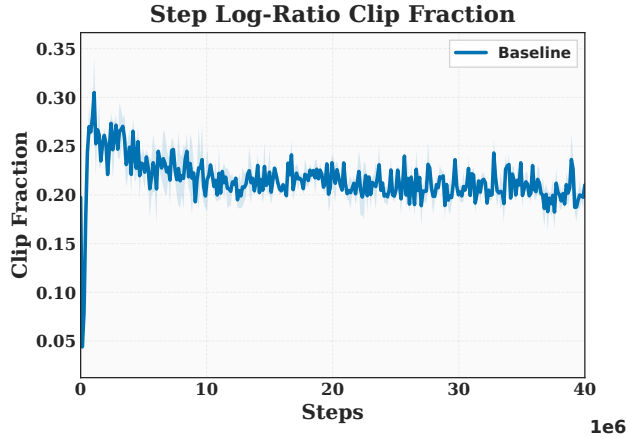


Figure 6: Fraction of rollout samples whose step log-ratio is clipped during GSB-MDPO training on HalfCheetah-v5. Results are averaged over two random seeds, with shaded regions indicating variability across seeds.

C.3 Computational Efficiency

We report representative computational cost on HalfCheetah-v5 in Table 3. All methods are measured with seed 0 for 5M environment interactions using the same hardware configuration as Appendix B.1. The reported wall-clock time is the end-to-end training time, and GPU memory is measured from the process tree during training. This experiment is intended as a lightweight efficiency comparison rather than a full benchmark-wide profiling study.

Table 3: Representative computational cost on HalfCheetah-v5 for 5M environment interactions. Wall-clock time is end-to-end training time. GPU memory is measured from the training process tree.

Method	Wall-clock	Rel. time	Peak GPU memory	Mean GPU memory
PPO	6.24 min	1.00×	0.60 GB	0.59 GB
FPO	6.96 min	1.12×	1.00 GB	0.97 GB
DPPO	8.67 min	1.39×	2.50 GB	2.38 GB
GenPO	28.73 min	4.61×	64.98 GB	64.13 GB
GSB-MDPO	7.39 min	1.19×	2.59 GB	2.51 GB

GSB-MDPO adds moderate overhead relative to Gaussian PPO due to storing denoising paths and recomputing current path likelihoods and drifts during policy updates. However, its end-to-end runtime remains close to FPO and lower than DPPO in this representative setting. Its GPU memory footprint is comparable to DPPO and substantially lower than GenPO, whose implementation requires much larger memory in this HalfCheetah-v5 run. These results suggest that the path-space objective can be implemented with practical overhead relative to other multi-step generative-policy baselines.